

Esercizi su Stream API

Esercizio 1 - Ordinamento

- **Data la classe**

```
public class Node {  
    private final String name;  
    public Node(String name) {  
        this.name = name;  
    }  
    public String name() {  
        return name;  
    }  
}
```

- **Ordina un elenco usando Collections.sort**
- **Implementa un comparatore usando lambda o un riferimento al metodo per ordinare i nodi lessicograficamente per nome. Ad esempio, generare la lista con**

```
List<Node> list = Arrays.asList(new Node("c"), new Node("b"), new Node("d"), new Node("a"));
```

- **e ordinarla**

Esercizio 2 - Divisione

- Dividi ogni numero di una lista di interi per 2 (ignorando il resto) e raccogli in un set.
- Applica a

```
List<Integer> numbers = asList(1, 2, 3, 4, 5);
```

Esercizio 3 - Stringa più lunga

- Trova la lunghezza della stringa più lunga
- Applica a

```
List<String> strings = asList("Lorem", "ipsum", "dolor", "sit", "amet,", "consectetur",  
"adipiscing", "elit.");
```

Esercizio 4 - Primi 5 numeri divisibili per 17

- Trova i primi 5 numeri divisibili per 17 generati da un flusso infinito
- Applica a

```
IntStream numbers = new Random(0).ints().map(Math::abs);
```

Esercizio 5 - Persone

- Hai una lista di persone, ciascuna rappresentata da un oggetto `Persona` con i seguenti attributi: nome, cognome, età e una lista di `String` rappresentanti le loro passioni.
- Obiettivo:
- Utilizza la `Stream API` per:
 1. Filtrare la lista e ottenere solo le persone maggiorenni.
 2. Ordinare le persone filtrate in base all'età in modo crescente.
 3. Raggruppare le persone per passione, creando una mappa dove la chiave è la passione e il valore è una lista di persone che condividono quella passione come prima nella loro lista.
 4. Stampare per ogni passione, il numero di persone interessate e la lista dei loro nomi e cognomi.

Esercizio 5 - Persone

```
import java.util.*;

public class Persona {
    private String nome, cognome;
    private int eta;
    private List<String> passioni;
    public Persona(String nome, String cognome, int eta, List<String> passioni){
        this.nome = nome;
        this.cognome = cognome;
        this.eta = eta;
        this.passioni = passioni;
    }
    public String getNome(){
        return nome;
    }
    public String getCognome(){
        return cognome;
    }
    public int getEta(){
        return eta;
    }
    public List<String> getPassioni(){
        return passioni;
    }
    public String toString(){
        return nome + " " + cognome;
    }
}
```

Esercizio 5 - Persone - Sfide aggiuntive

- **Calcolo statistiche:** Calcola l'età media delle persone per ogni passione.
- **Personalizzazione della stampa:** Formatta l'output in modo più elegante.

Esercizio 6 - Transazioni

- **Scenario:** Hai una lista di transazioni bancarie, rappresentate da una classe `Transazione` con attributi `data`, `importo` e `tipo` (ad esempio, "credito" o "debito").
- **Obiettivo:**
- **Utilizza la Stream API per:**
 1. Filtrare le transazioni effettuate in un determinato mese dell'anno.
 2. Calcolare l'importo totale dei crediti e dei debiti separatamente.
 3. Trovare la transazione con l'importo più alto in valore assoluto (sia per i crediti che per i debiti).
 4. Ordinare le transazioni in base alla data in ordine cronologico decrescente.

Esercizio 6 - Transazioni

```
import java.time.LocalDate;

public class Transazione {
    private LocalDate data;
    private double importo;
    private String tipo;

    public Transazione(LocalDate data, double importo, String tipo){
        this.data = data;
        this.importo = importo;
        this.tipo = tipo;
    }

    public LocalDate getData(){
        return data;
    }

    public double getImporto(){
        return importo;
    }

    public String getTipo(){
        return tipo;
    }

    public String toString(){
        return data + " , " + importo + " , " + tipo;
    }
}
```

Esercizio 6 - Transazioni

```
import java.util.*;
import java.util.stream.Collectors;
import java.time.*;

public class Esercizio6 {
    public static void main(String[] args){
        List<Transazione> transazioni = Arrays.asList(
            new Transazione(LocalDate.of(2023, Month.NOVEMBER, 15), 100.0, "credito"),
            new Transazione(LocalDate.of(2023, Month.APRIL, 11), 200.0, "debito"),
            new Transazione(LocalDate.of(2023, Month.AUGUST, 9), 300.0, "credito"),
            new Transazione(LocalDate.of(2023, Month.OCTOBER, 2), 400.0, "debito"),
            new Transazione(LocalDate.of(2023, Month.NOVEMBER, 8), 500.0, "credito")
        );
        // Punto 1: filtrare le transazioni effettuate in un determinato mese dell'anno

        // Punto 2: Calcolare l'importo totale dei crediti e dei debiti separatamente

        // Punto 3: Trovare la transazione con l'importo più alto in valore assoluto (crediti e debiti)

        // Punto 4: Ordinare le transazioni i nbase alla data in ordine cronologico
    }
}
```

Esercizio 7 - Prodotti

- Hai una lista di prodotti, ognuno rappresentato da una classe Prodotto con attributi nome, prezzo e categoria.
- Obiettivo:
- Utilizza la Stream API per:
 1. Filtrare i prodotti di una categoria specifica e con un prezzo superiore a una certa soglia.
 2. Trovare il prodotto più caro di ogni categoria.
 3. Calcolare il prezzo medio di tutti i prodotti.
 4. Creare una mappa dove la chiave è la categoria e il valore è una lista dei nomi dei prodotti in quella categoria.

Esercizio 7 - Prodotti

```
public class Prodotto {  
    private String nome, categoria;  
    private double prezzo;  
  
    public Prodotto(String nome, double prezzo, String categoria){  
        this.nome = nome;  
        this.prezzo = prezzo;  
        this.categoria = categoria;  
    }  
  
    public String getNome(){  
        return nome;  
    }  
  
    public double getPrezzo(){  
        return prezzo;  
    }  
  
    public String getCategoria(){  
        return categoria;  
    }  
  
    public String toString(){  
        return nome + " , " + prezzo + " , " + categoria;  
    }  
}
```

Esercizio 8 - Libri

- Hai a disposizione una classe Book con i seguenti attributi:

```
public class Book {  
    private String title;  
    private String author;  
    private int publicationYear;  
    private double price;  
    public Book(String title, String author, int publicationYear, double price) {  
        this.title = title;  
        this.author = author;  
        this.publicationYear = publicationYear;  
        this.price = price;  
    }  
    public String getTitle() {  
        return title;  
    }  
    public String getAuthor() {  
        return author;  
    }  
    public int getPublicationYear() {  
        return publicationYear;  
    }  
    public double getPrice() {  
        return price;  
    }  
    @Override  
    public String toString() {  
        return title + " by " + author + " (" + publicationYear + "), $" + price;  
    }  
}
```

Esercizio 8 - Libri

- Supponi di avere una lista di libri (`List<Book> books`) e di voler eseguire varie operazioni utilizzando la Stream API di Java. Scrivi il codice necessario per risolvere i seguenti punti:
- Filtro libri pubblicati dopo il 2010: Estrai una lista di libri pubblicati dopo il 2010.
- Ordinamento per prezzo: Ordina la lista di libri in ordine crescente di prezzo.
- Titoli di autori specifici: Crea una lista dei titoli dei libri scritti da un autore specifico (ad esempio, "J.K. Rowling").
- Media dei prezzi: Calcola la media dei prezzi di tutti i libri.
- Libro più costoso: Trova il libro con il prezzo più alto.
- Verifica prezzo massimo: Controlla se c'è almeno un libro che costa più di 30 euro.
- Stampa i libri ordinati per anno di pubblicazione in ordine decrescente.

Esercizio 9 - Analisi di una lista di persone

- Crea una classe **Persona** con i seguenti attributi:
- **nome (String)**
- **età (int)**
- **genere (enum Genere con valori MASCHIO e FEMMINA)**
- **città (String)**
- Crea una lista di oggetti **Persona** con almeno 10 persone, variando età, genere e città.

Esercizio 9 - Analisi di una lista di persone

- Utilizza la Stream API per risolvere i seguenti problemi:
 - a) Filtrare le persone che hanno più di 30 anni.
 - b) Estrarre i nomi delle persone che vivono in una città specifica (es. "Milano").
 - c) Calcolare l'età media delle persone di sesso femminile.
 - d) Trovare la persona più giovane nella lista.
 - e) Raggruppare le persone per città e stampare il risultato.
 - f) Verificare se tutte le persone della lista sono maggiorenni.
 - g) Contare quante persone vivono in una città specifica.